

Supabase

Backend as a Service — Auth, Database, Realtime, Storage

Supabase	PostgreSQL	Auth	RLS
----------	------------	------	-----

Realtime	Storage	Edge Functions
----------	---------	----------------

Parcours de formation — Développement d'Applications Mobiles

Djiguitech · Social Seller · Tous niveaux

■ Compétences visées

- ✓ Comprendre le positionnement de Supabase et ses différences avec Firebase
- ✓ Configurer et utiliser Supabase Auth pour l'authentification complète
- ✓ Implémenter Row Level Security (RLS) pour sécuriser les données multi-tenant
- ✓ Interroger la base de données via le client JavaScript et l'API REST
- ✓ Utiliser Supabase Realtime pour des interfaces temps réel
- ✓ Gérer des fichiers avec Supabase Storage

Table des matières

Chapitre 1 — Supabase — philosophie et architecture

- 1.1 Firebase vs Supabase
- 1.2 Architecture technique
- 1.3 Le client JavaScript
- 1.4 Dashboard et CLI

Chapitre 2 — Supabase Auth

- 2.1 Providers d'authentification
- 2.2 Sessions et JWT
- 2.3 Gestion des utilisateurs côté admin
- 2.4 Auth dans une app mobile

Chapitre 3 — Base de données et RLS

- 3.1 Accéder aux données
- 3.2 Filtrage, tri, pagination
- 3.3 Jointures avec `select()`
- 3.4 Row Level Security en profondeur

Chapitre 4 — Supabase Realtime

- 4.1 Postgres Changes
- 4.2 Subscribing to changes
- 4.3 Broadcast et Presence
- 4.4 Gestion des canaux

Chapitre 5 — Storage et Edge Functions

- 5.1 Buckets et politiques
- 5.2 Upload et download
- 5.3 Introduction aux Edge Functions

5.4 Déploiement en production

Supabase — philosophie et architecture

1.1 Firebase vs Supabase

Firebase (Google) et Supabase sont tous deux des plateformes BaaS (Backend as a Service) qui permettent de construire une application sans écrire de backend custom. Mais leur philosophie est fondamentalement différente. Firebase propose une base NoSQL (Firestore) et un écosystème propriétaire. Supabase est construit intégralement sur des technologies open source : PostgreSQL pour la base de données, PostgREST pour l'API REST auto-générée, GoTrue pour l'auth, Realtime basé sur les WAL de PostgreSQL.

Ce choix de l'open source a des implications importantes. Vous pouvez héberger Supabase sur vos propres serveurs (self-hosted). Vous utilisez le SQL standard de PostgreSQL — pas un langage de requête propriétaire. Vous bénéficiez de toute la puissance de PostgreSQL : RLS, fonctions stockées, triggers, JSONB, index. En cas de migration de Supabase vers un autre service, votre base de données PostgreSQL vous appartient entièrement.

Firebase vs Supabase — comparaison

Critère	Firebase	Supabase
Base de données	NoSQL (Firestore)	PostgreSQL (SQL)
Requêtes	SDK propriétaire	SQL + PostgREST + SDK
Temps réel	Listeners Firestore	Postgres WAL + WebSocket
Auth	Firebase Auth	GoTrue (JWT + OAuth)
Open Source	Non	Oui (self-hostable)
Typage	Limité	Génération TypeScript automatique

1.2 Architecture technique

Supabase orchestre plusieurs services open source derrière une API unifiée. PostgREST génère automatiquement une API REST à partir de votre schéma PostgreSQL — chaque table devient un endpoint. GoTrue gère l'authentification, l'émission de JWT, et les providers OAuth. Le service Realtime écoute le Write-Ahead Log (WAL) de PostgreSQL et diffuse les changements via WebSocket. Kong API Gateway fait office de proxy unifié devant tous ces services.

1.3 Le client JavaScript

La bibliothèque `@supabase/supabase-js` est le point d'entrée principal pour interagir avec Supabase depuis une application JavaScript. Elle expose une API chainable pour les requêtes de base de données, l'authentification, le stockage et le temps réel.

Initialisation du client Supabase

```
// Installation

npm install @supabase/supabase-js

// Initialisation (à faire une seule fois, dans un fichier dédié)

import { createClient } from '@supabase/supabase-js';

// Côté client mobile – clé ANON (publique)

export const supabase = createClient(
  process.env.EXPO_PUBLIC_SUPABASE_URL!,
  process.env.EXPO_PUBLIC_SUPABASE_ANON_KEY!
);

// Côté serveur Node.js – clé SERVICE_ROLE (contourne le RLS)

export const supabaseAdmin = createClient(
  process.env.SUPABASE_URL!,
  process.env.SUPABASE_SERVICE_ROLE_KEY!,
  { auth: { autoRefreshToken: false, persistSession: false } }
);
```

■■ Attention

Ne jamais utiliser la clé `SERVICE_ROLE` côté client (navigateur ou app mobile). Elle contourne tous les Row Level Security policies et donne un accès total à la base. Réservez-la exclusivement au code serveur s'exécutant dans un environnement sécurisé.

Supabase Auth

2.1 Providers d'authentification

Supabase Auth supporte de nombreux providers : email/password, magic link (email sans mot de passe), SMS OTP, et une trentaine de providers OAuth (Google, Apple, GitHub, etc.). Dans Social Seller, l'authentification par email/password est le provider principal, avec Supabase Auth gérant complètement le flux : inscription, connexion, vérification email, réinitialisation de mot de passe.

```
// Inscription

const { data, error } = await supabase.auth.signUp({
  email: 'user@example.com',
  password: 'secure-password-123',
  options: {
    data: { full_name: 'Mohamed Fomba' } // métadonnées user
  }
});

// Connexion

const { data: session, error } = await supabase.auth.signInWithPassword({
  email: 'user@example.com',
  password: 'secure-password-123',
});

// Déconnexion

await supabase.auth.signOut();

// Écouter les changements de session

supabase.auth.onAuthStateChange((event, session) => {
  if (event === 'SIGNED_IN') { /* rediriger vers l'app */ }
  if (event === 'SIGNED_OUT') { /* rediriger vers login */ }
});
```

2.2 Sessions et JWT

Supabase émet des JWT (JSON Web Tokens) lors de l'authentification. Le token d'accès expire après 1 heure par défaut et est automatiquement rafraîchi par le client. Le client Supabase peut persister la

session en stockage local (localStorage sur web, SecureStore sur mobile). En mobile, préférez expo-secure-store pour stocker les tokens de façon chiffrée dans le trousseau de l'OS.

2.3 Gestion des utilisateurs côté admin

L'API Admin de Supabase Auth permet de créer, modifier et supprimer des utilisateurs depuis le backend, sans nécessiter de mot de passe. Indispensable pour les tests automatisés : créer un utilisateur de test, tester les fonctionnalités, puis le supprimer — le tout programmatiquement.

```
// Côté serveur avec supabaseAdmin (SERVICE_ROLE)

// Créer un utilisateur sans mot de passe (pour les tests)
const { data, error } = await supabaseAdmin.auth.admin.createUser({
  email: `test-${Date.now()}@example.com`,
  email_confirm: true, // Pas d'email de confirmation
  user_metadata: { role: 'merchant' }
});

// Lister tous les utilisateurs
const { data: users } = await supabaseAdmin.auth.admin.listUsers();

// Supprimer un utilisateur (cascade sur les tables liées si configuré)
await supabaseAdmin.auth.admin.deleteUser(userId);
```


Base de données et RLS

3.1 Accéder aux données

PostgREST génère automatiquement une API REST complète depuis le schéma PostgreSQL. Le client JavaScript abstrait ces appels HTTP avec une API chainable. Chaque requête respecte les Row Level Security policies de la table ciblée.

```
// SELECT simple
const { data, error } = await supabase
  .from('products')
  .select('*')
  .is('deleted_at', null)
  .order('created_at', { ascending: false })
  .range(0, 19); // LIMIT 20 OFFSET 0

// INSERT
const { data, error } = await supabase
  .from('products')
  .insert({ name: 'Nouveau produit', price: 10000 })
  .select() // Retourner la ligne créée
  .single();

// UPDATE
const { error } = await supabase
  .from('products')
  .update({ name: 'Nouveau nom' })
  .eq('id', productId);

// DELETE (hard)
await supabase.from('products').delete().eq('id', productId);
```

3.2 Jointures avec select()

Supabase permet d'imbriquer des jointures directement dans la chaîne `select()` en utilisant les relations de clés étrangères définies dans le schéma. La syntaxe est intuitive : le nom de la table liée devient la clé, et vous listez les colonnes souhaitées.

```

// Commande avec items et produits imbriqués
const { data, error } = await supabase
  .from('orders')
  .select(`
    id,
    status,
    total_amount,
    customer_name,
    items:order_items (
      id,
      quantity,
      unit_price,
      product:products ( id, name, price )
    )
  `)
  .eq('id', orderId)
  .single();

// Conversations avec le dernier message
const { data } = await supabase
  .from('conversations')
  .select('*', messages(content, created_at, direction))
  .order('updated_at', { ascending: false });

```

3.3 Row Level Security en profondeur

Row Level Security (RLS) est la fonctionnalité la plus importante de Supabase pour la sécurité. Quand RLS est activé sur une table, PostgreSQL évalue une politique (policy) pour chaque ligne avant de la renvoyer ou de la modifier. Les politiques s'expriment en SQL et ont accès à la fonction `auth.uid()` qui retourne l'ID de l'utilisateur connecté.

Dans un SaaS multi-tenant, le pattern fondamental est : 'un utilisateur ne peut voir que les données de son tenant'. Cela se traduit par une politique qui compare le `tenant_id` de la ligne avec le `tenant_id` de l'utilisateur connecté, résolu via une sous-requête sur la table `users`.

Row Level Security — politique multi-tenant

```

-- Activer RLS sur la table
ALTER TABLE products ENABLE ROW LEVEL SECURITY;

-- Politique : un utilisateur voit seulement les produits de son tenant

```

```
CREATE POLICY 'tenant_isolation_products'
ON products
FOR ALL -- SELECT, INSERT, UPDATE, DELETE
USING (
tenant_id = (
SELECT tenant_id FROM users WHERE id = auth.uid()
)
);

-- Résultat : SELECT * FROM products retourne automatiquement
-- uniquement les produits du tenant de l'utilisateur connecté
-- sans aucun filtre explicite dans les requêtes côté client
```

■ Bonne pratique

Après avoir créé une politique RLS, testez-la avec un utilisateur réel (pas SERVICE_ROLE). Connectez-vous avec l'email d'un utilisateur du tenant A et vérifiez que vous ne voyez pas les données du tenant B. Cette vérification manuelle est indispensable avant tout déploiement en production.

Supabase Realtime

4.1 Postgres Changes

Supabase Realtime est un serveur WebSocket qui écoute le Write-Ahead Log (WAL) de PostgreSQL. Quand une ligne est insérée, modifiée ou supprimée, l'événement est diffusé à tous les abonnés en temps quasi-réel. C'est la technologie derrière les fonctionnalités de messagerie instantanée dans Social Seller.

Subscription Realtime dans un composant React

```
// S'abonner aux nouveaux messages d'une conversation
const channel = supabase
  .channel(`conversation-${conversationId}`)
  .on(
    'postgres_changes',
    {
      event: 'INSERT',
      schema: 'public',
      table: 'messages',
      filter: `conversation_id=eq.${conversationId}`,
    },
    (payload) => {
      const newMessage = payload.new as Message;
      setMessages(prev => [...prev, newMessage]);
    }
  )
  .subscribe();

// Nettoyage obligatoire (useEffect cleanup)
return () => {
  supabase.removeChannel(channel);
};
```

■ ■ Attention

Ne pas oublier de retirer le channel (removeChannel) quand le composant est démonté. Chaque channel non retiré maintient une connexion WebSocket ouverte et augmente la consommation mémoire de l'application.

Storage et Edge Functions

5.1 Buckets et politiques

Supabase Storage est un service de stockage de fichiers compatible S3. Les fichiers sont organisés en buckets (publics ou privés). Comme pour la base de données, des politiques RLS contrôlent qui peut lire, écrire ou supprimer des fichiers dans chaque bucket.

```
// Uploader un fichier

const { data, error } = await supabase.storage
  .from('logos') // nom du bucket
  .upload(`${tenantId}/logo.png`, file, {
    contentType: 'image/png',
    upsert: true, // Écraser si existant
  });

// Obtenir une URL publique

const { data: { publicUrl } } = supabase.storage
  .from('logos')
  .getPublicUrl(`${tenantId}/logo.png`);

// Obtenir une URL signée (bucket privé, expiration 1h)

const { data: { signedUrl } } = await supabase.storage
  .from('documents')
  .createSignedUrl(`${tenantId}/file.pdf`, 3600);
```

■ À retenir

- ✓ Supabase est construit sur PostgreSQL open source — vous gardez le contrôle total de vos données
- ✓ SERVICE_ROLE contourne le RLS — ne jamais l'exposer côté client
- ✓ RLS s'applique automatiquement : pas besoin de filtrer par tenant_id dans les requêtes client
- ✓ select() permet d'imbriquer les jointures grâce aux relations de clés étrangères
- ✓ Realtime écoute le WAL PostgreSQL — retirer les channels dans le cleanup du composant

—■ Exercices

Exercice 1 — Configuration complète

Configurer un projet Supabase from scratch pour une application de blog.

- Créer les tables : users, posts, comments avec les bonnes relations
- Activer RLS sur toutes les tables avec des politiques appropriées
- Tester les politiques : un utilisateur ne peut modifier que ses propres posts
- Générer les types TypeScript avec supabase gen types typescript

Exercice 2 — Auth flow complet

Implémenter un flux d'authentification complet dans une app React.

- Écran de login : email + password
- Persistance de session avec supabase.auth.onAuthStateChange
- Route protégée : rediriger vers login si non-connecté
- Bouton de déconnexion qui nettoie la session

Exercice 3 — Chat en temps réel

Construire un mini-chat avec Supabase Realtime.

- Table messages : id, content, user_id, created_at
- S'abonner aux INSERT avec filter sur un room_id
- Afficher les nouveaux messages sans rechargement
- Gérer le nettoyage du channel à la navigation

Bibliographie & Ressources

Documentation

- supabase.com/docs — documentation officielle
- supabase.com/docs/guides/auth/row-level-security — RLS en détail
- supabase.com/docs/reference/javascript — référence SDK JavaScript

Ressources

- supabase.com/blog — blog officiel avec guides avancés
- youtube.com/@Supabase — tutoriels vidéo officiels